ПРАКТИЧЕСКАЯ РАБОТА №1

Тема: Установка и настройка среды логического программирования Prolog.
Цель работы: Освоить установку и настройку инструментальных средств логического программирования (на примере SWI-Prolog / GNU Prolog / Visual Prolog), получить навыки работы с интерактивной средой (top-level), загрузки и выполнения простейших программ.

ЗАДАНИЕ:

1. Установка среды
   - Установить выбранную реализацию Prolog на рабочую машину.
   - Убедиться в работоспособности: запустить интерпретатор, проверить версию (?- version. - или аналог).
2. Настройка и знакомство с инструментарием
   - Изучить интерфейс: командную строку (top-level), встроенный редактор / связку с внешним редактором (VS Code + расширение, Emacs, встроенный IDE).
   - Настроить рабочую директорию для файлов программ.
   - Освоить базовые команды top-level: загрузка файла (`consult`, `[file].`), выход (`halt.`), справка (`help.`), трассировка (`trace.`, `notrace.`).
3. Написание первой программы
   - Создать файл с простой программой — базой фактов и правил (текст программы представлен в Приложении 1).
   - Загрузить программу в среду.
   - Выполнить серию запросов (примеры запросов в Приложении 2):
     проверка фактов;
     запросы с переменными;
     использование правил;
     проверка поведения при неуспехе и возврате (backtracking).
4. Подготовка отчета о выполненной работе
   - Оформить отчет в соответствии с предлагаемой структурой
   - Загрузить в эл. виде его на проверку в СЭО, дополнив (при наличии) файлом с программой.

Структура отчёта

Отчёт оформляется в текстовом и должен содержать:

1. Титульный лист
    - Название университета, кафедры, дисциплины.
    - Номер и тема работы.
    - ФИО студента, группа, ФИО преподавателя, дата.
2. Цель работы
3. Используемое ПО
    - Название и версия реализации Prolog.
    - ОС и параметры среды.
    - Краткое обоснование выбора (1–2 предложения).
4. Ход выполнения работы

    4.1. Установка и настройка.

    Скриншоты: запуск интерпретатора, вывод версии, окно редактора/IDE, рабочая директория.

    4.2. Освоение команд top-level.

    Перечень освоенных команд с примерами вызова и краткими пояснениями.

    4.3. Текст программы.

    Листинг файла с комментариями (в выбранном формате: SWI-Prolog / GNU Prolog / Visual Prolog) — см. Приложение 1.

    4.4. Результаты контрольных запусков.

    Привести несколько экранных форм (скриншоты) с демонстрацией работы программы по запросам из Приложения 2. Пояснить суть каждого запроса и полученный результат.
5. Анализ результатов

    Указать замеченные особенности среды (сообщения, предупреждения, поведение при ошибках).
6. Выводы
    - Что освоено, какие трудности возникли, как они решены.
    - Соответствие полученного результата поставленной цели.
7. Приложение (при необходимости)

ПРИЛОЖЕНИЕ 1. Текст минимальной программы для проверки среды

Ниже приведена минимальная программа «Родственные отношения», демонстрирующая базовые конструкции Prolog: факты, правила, унификацию и возврат. Программа представлена в трёх форматах — для SWI-Prolog, GNU Prolog и Visual Prolog.

SWI-Prolog (файл family.pl)

```
% ============================================
% Минимальная программа для проверки среды
% SWI-Prolog
% ============================================

% --- Факты: родительские отношения ---
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).
parent(pat, jim).

% --- Факты: пол ---
male(tom).
male(bob).
male(pat).
male(jim).
female(liz).
female(ann).

% --- Правила ---
father(X, Y) :- parent(X, Y), male(X).
mother(X, Y) :- parent(X, Y), female(X).

grandparent(X, Z) :- parent(X, Y), parent(Y, Z).

ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
```

Загрузка и запуск (*bash*):

```
swipl family.pl
```

или в интерактивном режиме:

```
?- [family].
```

GNU Prolog (файл family.pro)

```
% ============================================
% Минимальная программа для проверки среды
% GNU Prolog
% ============================================

% --- Факты: родительские отношения ---
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, pat).
parent(pat, jim).

% --- Факты: пол ---
male(tom).
male(bob).
male(pat).
male(jim).
female(liz).
female(ann).

% --- Правила ---
father(X, Y) :- parent(X, Y), male(X).
mother(X, Y) :- parent(X, Y), female(X).

grandparent(X, Z) :- parent(X, Y), parent(Y, Z).

ancestor(X, Y) :- parent(X, Y).
ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).
```

Загрузка и запуск (*bash*):

```
gprolog
```

или в интерактивном режиме:

```
| ?- [family].
```

Visual Prolog (файл family.pro)

Visual Prolog использует другой синтаксис:
обязательны секции `domains`, `predicates` и `clauses`.
Файл создаётся в среде Visual Prolog IDE.

```
% ============================================
% Минимальная программа для проверки среды
% Visual Prolog
% ============================================

domains
    name = symbol.

predicates
    parent(name, name).
    male(name).
    female(name).
    father(name, name).
    mother(name, name).
    grandparent(name, name).
    ancestor(name, name).

clauses
    % --- Факты: родительские отношения ---
    parent(tom, bob).
    parent(tom, liz).
    parent(bob, ann).
    parent(bob, pat).
    parent(pat, jim).
```

```
    % --- Факты: пол ---
    male(tom).
    male(bob).
    male(pat).
    male(jim).
    female(liz).
    female(ann).

    % --- Правила ---
    father(X, Y) :- parent(X, Y), male(X).
    mother(X, Y) :- parent(X, Y), female(X).

    grandparent(X, Z) :- parent(X, Y), parent(Y, Z).

    ancestor(X, Y) :- parent(X, Y).
    ancestor(X, Y) :- parent(X, Z), ancestor(Z, Y).

goal
    % Точка входа (при необходимости)
    console::write("Visual Prolog: family loaded successfully\n").
```

Создание проекта:

1. В *Visual Prolog IDE* выбрать ***Project → New***.
2. Указать тип проекта (***Console Application***).
3. Вставить код в соответствующие секции.
4. Собрать проект (***Build***) и запустить (***Run***).

ПРИЛОЖЕНИЕ 2. Рекомендуемые запросы для проверки

Приведённые ниже запросы предназначены для выполнения в интерактивной среде после загрузки программы. Они демонстрируют основные механизмы логического программирования.

2.1. Базовые запросы (проверка фактов и правил)

№	Запрос	Ожидаемый результат	Что демонстрирует
1	`parent(tom, bob).`	`true`	Проверка наличия факта
2	`parent(bob, tom).`	`false`	Отсутствие факта в базе
3	`male(tom).`	`true`	Проверка факта
4	`female(tom).`	`false`	Проверка отрицательного факта
5	`father(tom, bob).`	`true`	Работа правила (tom — отец bob)
6	`mother(liz, ann).`	`false`	Правило не выполняется (liz — не мать ann)

2.2. Запросы с переменными (поиск решений)

№	Запрос	Ожидаемый результат	Что демонстрирует
7	`parent(tom, X).`	`X = bob ; X = liz`	Перебор всех решений
8	`parent(X, pat).`	`X = bob`	Поиск родителя
9	`grandparent(X, jim).`	`X = bob`	Работа правила grandparent
10	`ancestor(tom, X).`	Несколько решений (bob, liz, ann, pat, jim)	Рекурсивное правило

2.3. Запросы, демонстрирующие *backtracking*

№	Запрос	Как выполнять	Что демонстрирует
11	`parent(tom, X).`	Нажать `;` для следующего решения	Механизм возврата
12	`ancestor(X, jim).`	`;` несколько раз	Поиск всех предков
13	`parent(X, Y).`	`;` для перебора всех пар	Полный перебор базы